

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DANIELA ARALDI

**DESENVOLVIMENTO DE AGENTE AUTÔNOMO PARA SUGESTÃO E
CRIAÇÃO DE REGRAS SIGMA PARA SIEMS**

PATO BRANCO

2025

DANIELA ARALDI

**DESENVOLVIMENTO DE AGENTE AUTÔNOMO PARA SUGESTÃO E
CRIAÇÃO DE REGRAS SIGMA PARA SIEMS**

**Autonomous Agent Development for Suggesting and Creating Sigma Rules
for SIEMs**

Projeto de Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia de Computação do Curso de Bacharelado em Engenharia de Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Me. Adriano Serckumecka

Coorientador: Prof. Dr. Jefferson Tales Oliva

PATO BRANCO

2025



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

RESUMO

O resumo deve ressaltar de forma sucinta o conteúdo do trabalho, incluindo justificativa, objetivos, metodologia, resultados e conclusão. Deve ser redigido em um único parágrafo, justificado, contendo de 150 até 500 palavras. Evitar incluir citações, fórmulas, equações e símbolos no resumo. A referência no resumo é elemento opcional em trabalhos acadêmicos, sendo que na UTFPR adotamos por não incluí-la nos resumos contidos nos próprios trabalhos. As palavras-chave e as keywords são grafadas em inicial minúscula quando não forem nome próprio ou nome científico e separados por ponto e vírgula.

Palavras-chave: segurança; monitoramento; inteligência artificial; sigma; ameaças.

ABSTRACT

Seguir o mesmo padrão do resumo, com a tradução do texto do resumo e referência, se houver, para a língua estrangeira (língua inglesa).

Keywords: security; monitoring; artificial intelligence; sigma; threats.

SUMÁRIO

1 INTRODUÇÃO

1.1 Motivação

A segurança cibernética contemporânea representa a base da estabilidade civilizatória global e isto se agrava pela evolução constante das ameaças digitais e complexidade das infraestruturas tecnológicas modernas. Os ataques cibernéticos alimentam um ciclo vicioso de vulnerabilidades, comprometendo desde a integridade de dados sensíveis até a continuidade de serviços essenciais, como por exemplo nas áreas da saúde, energia e finanças. Este ciclo demanda abordagens inovadoras diante dos desafios crescentes em face às sofisticações dos ataques e da expansão das infraestruturas tecnológicas heterogêneas (??).

Por infraestrutura heterogênea, entende-se um ambiente composto de diferentes tipos de tecnologias, dispositivos, sistemas operacionais, plataformas e arquiteturas que coexistem e interagem entre si. Tal heterogeneidade aumenta a complexidade de gestão e de segurança, pois cada componente possui vulnerabilidades específicas, demandando atualizações distintas e estratégias diferentes de proteção. Neste cenário, um único ponto fraco pode comprometer todo o ecossistema. Portanto, isso reforça a ideia de que a defesa cibernética precisa ser cada vez mais inovadora e adaptativa (?????).

Os Sistemas de Gerenciamento de Informações e Eventos de Segurança (SIEMs) têm se consolidado como pilares na detecção e resposta a ameaças cibernéticas, especialmente diante da sofisticação dos ataques e do volume exponencial de dados gerados — um exemplo são os *logs*, registros automáticos de eventos que ocorrem em sistemas, aplicativos, redes ou dispositivos, gerados diariamente dentro de uma empresa. SIEMs, de forma similar aos *firewalls*, IDSs (Sistemas de Detecção e Intrusão), IPSs (Sistemas de Prevenção de Intrusão) e *syslogs*, são baseados em regras para filtrar e detectar ameaças e invasões. A má notícia é que essas regras tradicionais de detecção nem sempre acompanham a evolução das técnicas maliciosas, apresentando limitações diante da necessidade de respostas mais rápidas e inteligentes (????).

Dentre as iniciativas que buscam suprir essa lacuna, destacam-se as regras *Sigma*, um formato aberto e descritivo voltado à criação de regras de detecção independentes de plataformas, o que facilita sua adoção em diferentes soluções de SIEMs. No entanto, a criação e manutenção manual dessas regras pode se tornar um gargalo, principalmente em ambientes com elevado volume de eventos de segurança (??).

O padrão *Sigma* permite a criação de regras de detecção em YAML — uma linguagem de serialização de dados, usada para representar informações estruturadas de forma simples e legível, sendo compartilháveis e convertíveis para linguagens como *Splunk SPL* ou *Elasticsearch Query* (linguagens de consulta nativas respectivamente do *Splunk* e *Elasticsearch*), otimizando a colaboração entre as equipes de segurança. No entanto, a geração manual dessas regras ainda é lenta e propensa a lacunas, especialmente diante de ameaças emergentes, como um

possível *ransomware* (tipo de *malware* que sequestra dados ou sistemas e exige resgate para devolvê-los ao usuário) ou *zero-day exploit* — ataque em que se explora uma falha de segurança desconhecida do *software*, *firmware* ou *hardware* quando a vulnerabilidade ainda não foi descoberta pelo desenvolvedor e não há correção disponível (????).

Com isso, é evidente que a aplicação de técnicas de inteligência artificial (IA) é uma ferramenta promissora na automatização da geração de regras *Sigma*, agilizando a detecção de comportamentos suspeitos ou maliciosos, considerando o tempo útil da elaboração de uma regra e a sua qualidade técnica.

Sendo assim, este trabalho propõe a implementação de um agente capaz de automatizar a criação de regras *Sigma*, a partir de vulnerabilidades e dados históricos de ataques — CVEs, indicadores de comprometimento (IOCs) e *logs* de incidentes. A ideia principal é a ferramenta estar habilitada para criar regras pontuais, com base em nova notícia sobre vulnerabilidade reproduzida na *internet*, CVE, CWE, de forma supervisionada pelo analista (nada será totalmente automático), além de frequentemente buscar e sugerir atualizações para as regras utilizadas pelo usuário conforme novas informações sejam publicadas, de forma a melhorar sua acurácia.

O agente utilizará técnicas de Processamento de Linguagem Natural (NLP) para extrair padrões de ameaças em textos não estruturados, como relatórios de ataques e artigos técnicos (??), e técnicas de aprendizado de máquina — mais precisamente LLMs (*Large Language Models*) para a geração autônoma de regras *Sigma*. O estudo de ??) apud ??) mostra que na integração entre o *ChatGPT* e o SIEM *Wazuh*¹ houve eficácia na análise de *logs*, detecção de padrões de ameaças e automatização de respostas de segurança. O *feedback* dos usuários indicou que a LLM é fácil de usar e eficaz no fornecimento de informações detalhadas sobre segurança, beneficiando principalmente os usuários com conhecimento limitado sobre o assunto. Esse tipo de capacidade sugere que LLMs podem ser adaptados para gerar regras *Sigma* autônomas a partir de dados de ameaças em tempo real.

1.2 Objetivos

1.2.1 Objetivo geral

Desenvolver um agente que automatize a criação de regras *Sigma* para SIEM, treinado com a integração de dados históricos de ataques cibernéticos (IOCs, CVEs, *logs* de incidentes) e vulnerabilidades, visando otimizar a detecção de ameaças em SIEMs e reduzir a demora entre a identificação de uma ameaça e sua mitigação.

¹ O *Wazuh* é uma solução de segurança de código aberto que possui funcionalidades de HIDS (*Host-based Intrusion Detection System*, Sistema de Detecção de Intrusão baseado em Host) e SIEM (??).

1.2.2 Objetivos específicos

- Criação de regras *Sigma* utilizando LLMs;
- Desenvolvimento de um agente autônomo para a geração de regras *Sigma*;
- Comparação dos resultados dos métodos utilizados;

1.3 Estrutura do trabalho

Este trabalho possui cinco capítulos, organizados de forma a conduzir o leitor do embasamento teórico à abordagem metodológica que será adotada. No primeiro capítulo, apresenta-se a introdução do tema, defendendo a motivação, os objetivos e a justificativa do estudo. O capítulo 2 disserta sobre conceitos necessários para se compreender o assunto do trabalho: sistemas SIEM, regras *Sigma* e LLMs — o que são, como se configuram na segurança da informação e como se relacionam entre si diante da mitigação de ameaças. Já o capítulo 3 discute trabalhos relacionados, oferecendo uma visão crítica sobre pesquisas e soluções correlatas. O capítulo 4 descreve os materiais e os métodos utilizados, detalhando as etapas do desenvolvimento da proposta em três fases. Por fim, são apresentadas as referências que fundamentam teoricamente o trabalho.

2 REFERENCIAL TEÓRICO

Neste capítulo será discorrido sobre os principais conceitos que justificam o escopo deste trabalho, como o que são e qual a importância dos SIEMs na área de segurança da informação, o que são as regras *Sigma*, o que são LLMs e como os assuntos se relacionam entre si.

2.1 Segurança da Informação e SOC

2.1.1 SIEM

Um Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês *Security Information and Event Management* (SIEM) funciona como o núcleo central de inteligência em ambientes de cibersegurança, agregando e correlacionando *logs* provindos de toda a infraestrutura de TI — como *firewalls*, sistemas de detecção de intrusão (IDS), todos os tipos de servidores, bancos de dados e entre outras inúmeras aplicações. Sua essência reside na capacidade de contextualizar eventos aparentemente desconexos, identificando padrões que ferramentas isoladas não detectariam. Conforme destacam ??), essa contextualização é crucial em ações automatizadas contra ameaças complexas e citam o exemplo de um ataque de *zero-day*, onde a correlação entre uma tentativa de força bruta no *Active Directory* (serviço de diretório da *Microsoft* que gerencia identidades, recursos e permissões dentro da rede interna) e a existência de tráfego suspeito no *firewall* pode disparar respostas imediatas.

Nos Centros de Operações de Segurança (SOCs), o SIEM assume um papel operacional indispensável. Ele consolida alertas de diferentes fontes e formatos, prioriza incidentes com base na criticidade e orquestra processos de resposta (*workflows*). ??) enfatizam ainda que, sem um SIEM, os SOCs perdem a capacidade de detectar fases avançadas de ataques, como por exemplo, de movimento lateral e escalção de privilégios, que se caracterizam pela movimentação do invasor por meio dos sistemas do alvo enquanto adquire privilégios de acesso —visando o nível de administrador; é citado o caso da rede de hotéis *Starwood*, que permaneceu quatro anos sem identificar uma violação sequer e sofreu um vazamento de dados de 383 milhões de hóspedes ao redor do mundo em 2019 (??). Diante do exposto, essa visão unificada transforma o SIEM no “sistema nervoso” do SOC, permitindo que analistas investiguem ameaças sistêmicas, não apenas alertas isolados.

A eficácia de um SOC está intrinsecamente ligada à maturidade do SIEM. Regras mal calibradas geram excesso de falsos positivos; SIEMs não otimizados podem produzir mais de mil alertas diários, sobrecarregando as equipes — segundo ??), uma pessoa sozinha é capaz de analisar apenas cerca de 5 a 15 alertas por dia, a depender da criticidade dos mesmos. Por outro lado, uma configuração refinada permite que um único analista investigue milhares

de *logs*, elevando exponencialmente a eficiência operacional. A empresa Exabeam reforça que essa otimização é a base para um SOC ágil e proativo (??).

A importância estratégica de um SIEM convencional na cibersegurança moderna se ancora na sua capacidade de reduzir drasticamente o tempo de detecção de ameaças, principalmente porque, conforme o passar dos anos, qualquer tipo de dado se tornou o recurso mais valioso que existe. ?? citam um relatório de vazamentos de dados de 2018 da Verizon (??), informando que cerca de 68% das violações são descobertas meses (ou mais) após o ataque inicial; o último relatório, de 2025, não atualiza esta porcentagem, mas informa que o tempo médio de remediação de vulnerabilidades nos dispositivos de borda (especialmente falhas de *zero-day*) é de 32 dias (??). Este tipo de lacuna pode ser preenchida quando se tem um SIEM bem implementado, em virtude da correlação dos eventos em tempo real. Além disso, ele serve como alicerce para a evolução tecnológica: plataformas SIEM convencionais são a base necessária para a integração de IA e aprendizado de máquina, que posteriormente amplificam a detecção de ameaças sutis e permitem a automação de respostas (??).

»»Qual a diferença entre um SIEM e um antivírus tipo o Acronis?««««

Economicamente, embora o SIEM convencional envolva custos elevados (licenças baseadas em volume de *logs*, equipe especializada e SOC ininterrupto), ?? justificam seu investimento frente ao cenário de riscos globais: projetou-se US\$ 5,2 trilhões em perdas por ciberataques entre 2019 — 2023. Para organizações com dados sensíveis ou requisitos de conformidade, como o GDPR — Regulamento Geral sobre a Proteção de Dados da União Europeia, ou a PCI-DSS — *Payment Card Industry Data Security Standard*, norma globalmente reconhecida aplicada à segurança de dados de cartões de pagamento, o SIEM é não apenas viável, mas indispensável; sua ausência representa um lapso operacional intransponível, mesmo com outras soluções de segurança.

Em suma, o SIEM convencional se mantém como pilar irrevogável da segurança corporativa; não por sua tecnologia isolada, mas também por habilitar ecossistemas integrados de detecção e resposta. Conclui-se que sua implementação estratégica, ainda que desafiadora, é um divisor de águas entre organizações expostas a violações prolongadas e aquelas capazes de antecipar, conter e remediar ameaças em escala evolutiva.

2.1.2 SIEMs Inteligentes

A integração de aprendizado de máquina (do inglês, *machine learning*, ML) e inteligência artificial (IA) em SIEMs transforma a cibersegurança ao superar as limitações das abordagens baseadas em regras estáticas. De acordo com ??, a IA generativa consegue criar dados sintéticos para simular ameaças inéditas, permitindo a detecção proativa de ataques *zero-day* sem depender exclusivamente de dados previamente rotulados. Essa capacidade é amplificada pelos algoritmos de ML que analisam padrões contextuais em grandes volumes de *logs*, correlacionam eventos aparentemente desconexos (como tráfego anômalo com tentativas de força

bruta) e identificam ameaças complexas. A predição de ameaças e a automatização dessas análises são fatores críticos para a atuação dos SOCs.

A integração de técnicas de IA e aprendizado de máquina em sistemas SIEM melhora significativamente a correlação de grandes volumes de eventos em tempo real, reduzindo falsos positivos ou falsos negativos, acelerando a triagem de alertas (??). Conforme demonstrado por ??), algoritmos de detecção de anomalias, análise preditiva e limiares adaptativos permitem identificar proativamente atividades sutis ou ocultas que escapariam sob regras estáticas ou análise manual. Ainda, ao empregar ambos modelos supervisionados e não supervisionados, o SIEM com ML aprende continuamente a partir dos dados de *logs* e do comportamento da rede, ajustando-se dinamicamente às mudanças no ambiente e antecipando novos vetores de ataque (??).

É visto que, com o crescimento exponencial de dados e o aumento da complexidade das ameaças, é inviável depender exclusivamente de analistas humanos, tornando substancial o uso estratégico de ML/IA por parte dos SIEMs. Essa prática permite que o SIEM evolua de uma abordagem baseada em regras para uma abordagem comportamental e baseada em risco, onde perfis de usuários e entidades são criados automaticamente e usados para detectar desvios. Isso se alinha com o trabalho de ??), que propõem modelos de IA generativa para analisar fluxos de dados e gerar respostas simuladas a possíveis ataques, ajudando a antecipar e mitigar riscos de forma mais eficaz. Portanto, a incorporação de IA e ML em SIEMs representa um avanço significativo para a cibersegurança moderna, trazendo precisão, adaptabilidade e automação a um cenário cada vez mais dinâmico e hostil.

2.2 Bases de Padrões de Ataques

Para a criação de regras em SIEMs, assim como para qualquer tipo de regra, há de se adotar algum padrão. No mundo da cibersegurança existem padrões que catalogam e identificam vulnerabilidades, eventos e outras categorias do gênero. Duas categorizações são o CVE e o CWE. O CVE — *Common Vulnerabilities and Exposures* (Vulnerabilidades e Exposições Comuns), é um programa mantido pelo MITRE desde 1999, que define e cataloga vulnerabilidades e exposições de segurança publicamente conhecidas, no qual se atribui um identificador único (CVE-ID) e padrão para cada uma. Atualmente possuindo quase 300 mil registros, o CVE é uma referência confiável para integrar bancos de dados de segurança, alertas, produtos comerciais e soluções automatizadas (????).

Já o CWE — *Common Weakness Enumeration* (Enumeração de Fraquezas Comuns), também mantido pelo MITRE, é um catálogo comunitário de tipos de fraquezas (como falhas de validação, *buffer overflow*, injeção), usadas para identificar a causa raiz subjacente às vulnerabilidades. Quando um CVE descreve um “*buffer overflow*”, ele pode ser correlacionado a um CWE específico, permitindo que os analistas entendam quando se trata de um erro de lógica, design ou implementação (????).

O MITRE, que mantém as duas listagens citadas acima, é uma organização sem fins lucrativos, mantida por agências como o NIST (Instituto Nacional de Padrões e Tecnologia, do inglês *National Institute of Standards and Technology*) e CISA (Agência de Segurança Cibernética e Infraestrutura, do inglês *Cybersecurity and Infrastructure Security Agency*). Além da contribuição citada acima, o MITRE criou uma solução alternativa chamada MITRE ATT&CK — uma base de dados sobre táticas e técnicas adversárias, acessível globalmente, baseada em observações do mundo real. É usada como base para o desenvolvimento de modelos e metodologias de ameaças no setor privado, governo e na comunidade de produtos e serviços de cibersegurança (??). A base é aberta e está disponível gratuitamente para uso de qualquer indivíduo ou organização, assim como o CVE e CWE.

Não menos importante é o NVD — *National Vulnerability Database* ou Base Nacional de Dados de Vulnerabilidades, repositório do governo dos EUA e mantido pelo NIST. É responsável por armazenar dados padronizados sobre vulnerabilidades de segurança seguindo o protocolo SCAP (*Security Content Automation Protocol*). O repositório inclui listas de falhas em *softwares*, métricas de impacto, nomes de produtos, configurações vulneráveis e *checklists*, permitindo automação da gestão de vulnerabilidades, avaliação de segurança e conformidade (??).

Esses elementos são pilares para as regras de SIEMs/*Sigma*, pois fornecem contexto de acionamento e padronização. O NVD enriquece CVEs com dados de severidades (como o CVSS (*Common Vulnerability Scoring System*), método usado para fornecer uma medida qualitativa de gravidade (??)) e produtos afetados, permitindo que SIEMs priorizem alertas com base no risco real. As classificações CWE ajudam a desenvolver regras *Sigma* para fraquezas comuns, detectando ataques mesmo quando variantes novas surgem. Já o mapeamento MITRE entre CWE e CVE vincula falhas de código a vulnerabilidades exploradas, refinando a correlação de ameaças em SIEMs. Por exemplo, uma regra *Sigma* pode usar o CWE-352¹ para monitorar solicitações não autorizadas, enquanto o CVE associado direciona *patches* críticos (??).

2.3 Regras *Sigma*

Sigma (*Generic Signature Format for SIEM Systems*, “Assinatura em formato genérico para sistemas SIEM” em tradução livre (??)) é o projeto de código aberto de um padrão genérico de descrição de *logs* de eventos, aplicável a qualquer tipo de *log*, com o intuito de compartilhar a lógica de detecção de ameaças ou anomalias entre diferentes SIEMs. Ao operar com dados de ameaças de múltiplas fontes, o *Sigma* capacita os caçadores de ameaças a correlacionar eventos que passariam despercebidos por abordagens convencionais. Dessa forma, se obtém conhecimentos estratégicos sobre dados confiáveis e antecipa-se tentativas de ataque em estágios iniciais desta cadeia de eventos (??).

¹ Falsificação de solicitação entre sites, do inglês *Cross-Site Request Forgery* (CSRF) (??).

Os mantenedores da iniciativa consideram que excelentes análises podem ser encontradas na *internet*, incluindo IOCs e regras YARA² para detectar arquivos ou ferramentas maliciosos, por exemplo, mas que muitas vezes limitam-se a descrições específicas, funcionais para poucos *softwares*. Portanto, da necessidade de haver um método de descrição específico ou genérico de eventos de *logs*, com portabilidade para vários SIEMs, a fim de aprimorar os recursos de detecção para todos, surgiu o Sigma (??).

O projeto de código aberto foi criado em 2017 por Florian Roth e Thomas Patzke (??), inspirados em padrões já existentes nas áreas de análise de *malware* e monitoramento de rede. Como já dito, o padrão *Sigma* se diferencia por lidar com vários tipos de esquemas de *logs*, unindo padrões que já existem e se diferenciam entre si (??).

O projeto carrega essa denominação pois se considera um ecossistema — envolve três componentes: o formato *Sigma*, a ferramenta *Sigma* e a coleção de regras *Sigma*. Este último diz respeito ao repositório no *Github*, que possui mais de 3 mil regras de detecção de diferentes tipos; por ser de código aberto, pode receber sugestões de qualquer pessoa disposta a contribuir (??). Já a ferramenta *Sigma*, refere-se a um conversor de regras disponível para *download* — o *Sigma Command Line Interface* (CLI). Com ele é possível converter as regras *Sigma* para as regras usadas no SIEM de interesse do usuário; *Elasticsearch*, *CrowdStrike*, *IBM QRadar AQL* e *Splunk* são algumas marcas registradas para as quais essa integração está disponível.

2.3.1 Sigma e a mitigação de ameaças

As regras *Sigma* formam um conjunto estruturado de princípios e métodos essenciais para o sucesso da investigação focada em determinar as ameaças que podem prejudicar um ou mais sistemas (*Threat Hunting*), oferecendo um padrão uniforme que orienta os profissionais de segurança na elaboração de regras criteriosas para detecção de ameaças ou anomalias. Segundo (??), essas regras otimizam a identificação, avaliação e a resposta a incidentes, proporcionando decisões mais precisas e oportunas, além de uma base sólida para controlar vetores de ameaças, minimizar riscos e prevenir violações de dados. Dessa forma, as regras *Sigma* constituem o alicerce de uma estratégia de segurança proativa e resiliente, capaz de antecipar e mitigar possíveis ataques de forma sustentável.

A caça a ameaças é um processo defensivo proativo que visa descobrir e neutralizar invasores, mapeando vulnerabilidades e antecipando movimentos adversos por meio da análise contínua de comportamentos anômalos e incidentes em curso. Nesse cenário, as regras *Sigma*, desde sua padronização, em 2017, desempenham um papel fundamental ao oferecer uma linguagem comum para correlação de eventos em múltiplas plataformas SIEM, unificando dados de sistemas heterogêneos e acelerando a detecção de padrões suspeitos. Conforme demonstrado por (??), essa unificação reduz significativamente a carga de trabalho manual dos

² Uma regra YARA é um arquivo de extensão “.yar” contido de expressões lógicas usadas para classificar e identificar amostras de *malware* (??).

analistas, permitindo identificar ameaças ainda na fase inicial de comprometimento, elevando a eficácia das operações de *Threat Hunting*, ou “caça a ameaças”.

No entanto, ainda que a aplicação das regras *Sigma* viabilize inúmeros cenários benquistos, vale ressaltar que, ainda assim, os SIEMs continuarão sendo heterogêneos, pois utilizam formatos e padrões de dados diferentes — situação inclusive de eventos de diferentes dispositivos. As regras *Sigma* surgem como uma forma de expressar a regra ou a ameaça e o que deve ser buscado, mas ainda precisa ser convertida para o SIEM de destino (através do CLI), que já possui suas próprias regras.

2.3.2 Estrutura

As regras *Sigma* são arquivos YAML contidos de informações que viabilizam a detecção de comportamento estranho ou malicioso quando da inspeção de arquivos de *logs* dentro do contexto de um SIEM (??).

A compreensão da estrutura de uma regra *Sigma* e como é usada para detecção faz-se importante em razão de que parte da metodologia envolverá selecionar as informações mais críticas das regras e avaliá-las.

As regras são divididas em três grupos principais de instruções (??):

- Detecção (*detection*): qual comportamento malicioso a regra está procurando;
- Fonte do log (*logsource*): quais tipos de logs a detecção deve procurar; e
- Metadados (*title*, *id*, *status*, etc.): detalhes da regra.

A Figura ?? é um exemplo de regra *Sigma* que está disponível na página oficial do projeto.

A seção de detecção é o coração da regra *Sigma*, pois especifica o alvo da regra no vasto volume de *logs*. Adiante neste trabalho este componente será mencionado como um dos principais para a análise da qualidade das regras geradas pelas IAs. Vale mencionar que, dentro da seção de detecção, existem instruções para a escrita, assim como qualquer linguagem de programação; no entanto, num primeiro momento, basta compreender o esqueleto principal. Sendo assim, a segunda peça mais importante da regra é a fonte dos *logs*, o campo *logsource*; nele se especifica qual o tipo de *log* deve ser canalizado pela regra — ao invés de varrer todos os *logs* do SIEM. Por fim, as demais informações, os metadados, servem para guiar o usuário sobre o assunto da regra (??).

Para fins de orientação quando da criação de uma nova regra, é disponibilizado um esqueleto da regra com instruções (Figura ??) no repositório *Sigma* (??).

Figura 1 – Exemplo de regra *Sigma* (??)

```

1  # ./rules/cloud/okta/okta_user_account_locked_out.yml
2  title: Okta User Account Locked Out
3  id: 14701da0-4b0f-4ee6-9c95-2ffb4e73bb9a
4  status: test
5  description: Detects when a user account is locked out.
6  references:
7    - https://developer.okta.com/docs/reference/api/system-log/
8    - https://developer.okta.com/docs/reference/api/event-types/
9  author: Austin Songer @austinsonger
10 date: 2021-09-12
11 modified: 2022-10-09
12 tags:
13   - attack.impact
14 logsource:
15   product: okta
16   service: okta
17 detection:
18   selection:
19     displaymessage: Max sign in attempts exceeded
20     condition: selection
21 falsepositives:
22   - Unknown
23 level: medium

```

Fonte: Captura de tela da página *Sigma Rules* em ??).

2.4 Processamento de Linguagem Natural

Segundo ??), o Processamento de Linguagem Natural (PLN) é definido como uma abordagem computacional teórica e tecnológica fundamentada para analisar textos em linguagem humana. Seu objetivo central é alcançar um processamento linguístico semelhante ao humano, através da representação de textos aos níveis linguísticos: fonológico (padrões de entonação e fonemas), morfológico (estrutura das palavras), léxico (significado individual das palavras), sintático (estrutura gramatical das frases), semântico (significado literal das frases), discursivo (coesão e estrutura dos textos) e pragmático (contexto); ou seja, o PLN fragmenta o texto e analisa sistematicamente as sete camadas citadas (????).

As técnicas de PLN evoluíram de métodos simbólicos, baseados em regras e lógica formal, para abordagens estatísticas e conexionistas (ou seja, inspirada no funcionamento do cérebro humano), impulsionadas pelos avanços computacionais e a disponibilidade de coleções extensas e estruturadas de textos (??).

O PLN suporta aplicações críticas como a Recuperação de Informação (RI), Extração de Informação (EI), Tradução Automática (TA) e Sistemas de Pergunta-Resposta. ??) cita um exemplo de RI em que sistemas PLN melhoraram a precisão em representar consultas e documentos em diversos níveis semânticos. A EI utiliza reconhecimento de entidades e análise parcial para estruturar dados de textos não estruturados, como em anúncio de emprego. Já os sistemas de diálogo, embora ainda dependentes de dimensões fonético-léxicas, buscam incor-

Figura 2 – Exemplo de regra *Sigma* (??)

```

1 title: a short capitalised title with less than 50 characters
2 id: generate one here https://www.uuidgenerator.net/version4
3 status: experimental
4 description: A description of what your rule is meant to detect
5 references:
6 | - A list of all references that can help a reader or analyst understand the meaning of a triggered rule
7 tags:
8 | - attack.execution # example MITRE ATT&CK category
9 | - attack.tl059 # example MITRE ATT&CK technique id
10 | - car.2014-04-003 # example CAR id
11 author: Michael Haag, Florian Roth, Markus Neis # example, a list of authors
12 date: 2018/04/06 # Rule date
13 logsource: # important for the field mapping in predefined or your additional config files
14 | category: process_creation # In this example we choose the category 'process_creation'
15 | product: windows # the respective product
16 detection:
17 | selection:
18 | | FieldName: 'StringValue'
19 | | FieldName: IntegerValue
20 | | FieldName|modifier: 'Value'
21 | condition: selection
22 fields:
23 | - fields in the log source that are important to investigate further
24 falsepositives:
25 | - describe possible false positive conditions to help the analysts in their investigation
26 level: one of five levels (informational, low, medium, high, critical)|

```

Fonte: Captura de tela do código genérico na página *Rule Creation Guide* no repositório Sigma(??).

porar todas as camadas linguísticas para resultados mais naturais. A sumarização automatizada — geração de resumos concisos e coerentes a partir de textos extensos, aplica análise discursiva para condensar os textos enquanto mantém coerência (????).

Alguns dos obstáculos do PLN envolvem a integração de conhecimento de mundo e raciocínio de senso comum, essenciais para resolver ambiguidades contextuais complexas. Avanços nos modelos híbridos e ferramentas aceleram os experimentos, mas a compreensão profunda permanece distante, pois exige bases de conhecimento amplas e técnicas como inferência abduativa para superar lacunas de interpretação (??).

2.4.1 Modelos de Linguagem de Grande Escala (LLMs)

Os Grandes Modelos de Linguagem, do inglês *Large Language Models* (LLMs), são sistemas de inteligência artificial treinados com bilhões ou trilhões de parâmetros, capazes de compreender e gerar textos em linguagem natural, ou seja, no formato de comunicação humano. A arquitetura desses modelos é baseada em “*Transformers*” — redes neurais baseadas em mecanismos de *self-attention* que permitem que o modelo foque em diferentes partes do texto simultaneamente, adquirindo relações contextuais de longo alcance entre as palavras (??); estes aprendem em grande escala através do aprendizado auto supervisionado (*self-supervised learning*), extraindo padrões de grandes corpos de textos sem rótulos explícitos (??). A evolução dessas redes marca um avanço na capacidade das máquinas “entenderem” contexto e intenção.

» » » » » » » » » » ??) explica que o mecanismo central de um LLM se baseia na predição de *tokens* (menores unidades de texto que um modelo de linguagem divide para processar) sequenciais: dada uma sequência de entrada, o modelo estima a probabilidade do próximo elemento textual, refinando essa habilidade ao longo do pré-treino. Além disso, explica que técnicas como a “*in-context learning*” permitem que o mesmo modelo execute tarefas específicas apenas com os exemplos fornecidos pelo *prompt*, sem haver necessidade de ajuste fino. Esse paradigma torna possível a aplicação do mesmo modelo de LLM em diferentes tarefas sem precisar retreiná-lo (??).

» » » » » » » » » » > A aplicabilidade dos LLMs é vasta: auxiliam na geração de código, criação de conteúdos, atendimento ao cliente, análise de sentimentos e suporte à decisão clínica, entre outras funções (**??**). Contudo, desafios sempre existirão, e o alto custo computacional é um deles, além da tendência de reprodução de vieses presentes nos dados originais e vulnerabilidade a ataques de *jailbreak*³ e injeção de *prompt*⁴. Superar qualquer entrave exigirá trabalho contínuo, transparência dos dados e aperfeiçoamento incessante das técnicas de mitigação de riscos.

2.4.2 Arquitetura Transformer

2.4.3 Engenharia de Prompt

2.5 Geração Aumentada por Recuperação (RAG)

2.5.1 *Embeddings* e Bancos de Dados Vetoriais

2.5.2 RAG aplicado à Cibersegurança

2.6 Agentes Autônomos

2.6.1 LangGraph como Orquestrador

³ Processo de exploração de falhas de um dispositivo eletrônico bloqueado para instalar outro *software* que não o disponibilizado pelo fabricante para uso no dispositivo (??).

4 Ocorre quando *prompts* do usuário alteram o comportamento ou a saída do LLM de maneiras não intencionais. Essas entradas podem afetar o modelo mesmo que sejam imperceptíveis para humanos. Portanto, as injeções de *prompt* não precisam ser visíveis/legíveis para humanos, desde que o conteúdo seja analisado pelo modelo. (??)

3 TRABALHOS RELACIONADOS

3.1 Machine Learning Algorithms in SIEM Systems for Enhanced Detection and Management of Security Events

O estudo de ??) propõe a integração de algoritmos de *machine learning* (ML) em SIEMs convencionais para superar insuficiências na detecção das ameaças modernas. O objetivo é aprimorar a identificação de ataques desconhecidos e reduzir falsos positivos por meio de um modelo dinâmico que analisa padrões históricos. Para validar a abordagem, os pesquisadores implementaram uma arquitetura utilizando a pilha ELK ¹ e simularam cenários reais de ciberataques na ferramenta DVWA (*Damn Vulnerable Web Application*), criada especialmente para testes de penetração. Um modelo de CNN (Rede Neural Convolucional) foi treinado com dados de amostras de ataques do repositório MITRE, priorizando a detecção de *ransomware* e movimentação lateral. Os autores também mencionam que o código foi desenvolvido para comparação com modelos de *Random Forest* (algoritmo de ML que combina a saída de múltiplas árvores de decisão para alcançar um único resultado (??)) e SVM (*Support vector machine* — algoritmo supervisionado de ML que classifica dados encontrando uma linha ou hiperplano ótimo que maximiza a distância entre cada classe em um espaço N-dimensional (??)).

Os autores afirmam ganhos operacionais significativos nos seus resultados: o sistema reduziu falsos positivos ao correlacionar eventos de múltiplas fontes (*firewalls*, IDS e servidores), automatizou respostas em tempo real (como bloqueio de IPs via integração com *bots* do *Telegram*) e otimizou a eficiência de SOCs ao transformar alertas brutos em ações acionáveis. ??) concluíram que os modelos de ML identificam padrões sutis em grandes volumes de *logs* com maior precisão do que as regras estáticas, além de se adaptarem continuamente a novas táticas de ataque sem intervenção manual constante. A pesquisa contribui para entender um caminho de integração entre ML e SIEM, neste caso utilizando técnicas de aprendizado não supervisionado; no entanto, não fornece números quantitativos específicos.

A solução do trabalho conecta-se com o presente projeto através do objetivo de ambos, que é sofisticar os SIEMs para o melhor enfrentamento de novas ameaças e/ou prevenção de ataques, e da base de vulnerabilidades em comum — padrão MITRE. Os trabalhos diferem-se principalmente na abordagem: ??) possuem uma metodologia mais qualitativa e descritiva, servindo de exemplo teórico. Já o trabalho da autora é um estudo qualitativo e quantitativo, de teor empírico, que fornecerá percentuais de dados comparativos no final.

¹ Sigla para três ferramentas de código aberto: *Elasticsearch*, que fornece funcionalidades de análise e pesquisa; *Logstash*, responsável por coletar, agregar e armazenar dados usados pelo *Elasticsearch*; e *Kibana*, que fornece a interface do usuário e informações sobre os dados coletados e analisados anteriormente pelo *Elasticsearch*.

3.2 Automated Log Analysis and Anomaly Detection Using Machine Learning

O estudo de ??), tem como objetivo propor um modelo automatizado para a detecção de anomalias em arquivos de *logs*, enfrentando o desafio da ausência de dados rotulados. A motivação central do trabalho é reduzir a sobrecarga de alertas nos sistemas de segurança e eliminar a necessidade de inspeção manual extensiva, inviável em ambientes com grandes volumes de *logs*. Para isso, os autores buscaram uma solução que aproveita tanto técnicas de aprendizado de máquina quanto o conhecimento prévio dos especialistas.

A metodologia do trabalho combina aprendizado não supervisionado (*K-means* para *clustering*) com aprendizado supervisionado (classificação com *Naive Bayes*), utilizando regras automatizadas geradas a partir de conhecimento especializado para rotular registros inicialmente não estruturados. Após a etapa de pré-processamento e vetorização de dados com TF-IDF, os autores aplicam *Elbow Method* para definir o número ideal de *clusters* e, em seguida, criam regras baseadas em padrões observados nos *clusters* para construir um conjunto de dados rotulados. Esse conjunto, então, é usado para treinar um classificador que distingue eventos normais e anômalos, alcançando acurácia de 98% na base de testes.

O artigo de ??) propõe a geração automática de rótulos com base em regras para prever ameaças com a ajuda de ML. Já o presente trabalho busca desenvolver um agente autônomo que crie regras *Sigma* para sistemas SIEM. Os autores do estudo concluem que a detecção automatizada de anomalias enfrenta alguns desafios e um deles é a heterogeneidade dos ambientes de monitoramento e igualmente seus rótulos — desafio este que se pretende minimizar com a automatização da criação de regras *Sigma*.

Enquanto ??) foca em rotular automaticamente dados de *logs*, o uso das regras *Sigma* permite a unificação dos padrões, amenizando os desafios dos ambientes heterogêneos, mencionado pelos autores. Dentre outros pontos, este trabalho propõe uma solução mais genérica, com regras prontas para implantação, enquanto o trabalho citado aborda um modelo de classificação focado em *logs*.

3.3 From Text to MITRE Techniques: Exploring the Malicious Use of Large Language Models for Generating Cyber Attack Payloads

O artigo de ??) investiga os riscos associados ao uso malicioso de modelos de linguagem generativa para criação de códigos ofensivos baseados nas técnicas do MITRE ATT&CK. O objetivo do estudo foi avaliar a capacidade desses modelos em gerar, via engenharia de *prompt*, códigos executáveis correspondentes às 10 técnicas MITRE mais usadas por atacantes em 2022. Também buscam comparar o desempenho do *ChatGPT* e *Bard* (nomenclatura antiga do *Google Gemini*) em termos de geração, eficácia e sua habilidade de explicar claramente o que o código faz, como funciona e qual seu impacto.

Os autores aplicaram engenharia de *prompt* em ambos modelos já citados, buscando contornar restrições éticas e técnicas impostas por padrão para as inteligências, com o intuito de gerar com sucesso *scripts* maliciosos. Dois exemplos são as técnicas T1055, de Injeção de processos (??), e T1486, de Criptografia para Impacto (??). Esses códigos foram executados em um ambiente controlado (*sandbox*), permitindo que a análise dos resultados ocorresse em segurança. O estudo de ??) se baseou em dados do *Red Report* de 2023 (??) da *Picus Labs* — divisão de pesquisa e desenvolvimento da empresa de segurança cibernética *Picus Security* (??), que analisou mais de 500 mil amostras de *malware* reais. A análise demonstrou que o *ChatGPT* foi mais eficaz do que o *Google Bard* na geração de código funcional e correção de erros.

O artigo destaca tanto os riscos quanto os possíveis usos defensivos da tecnologia, como a simulação de cenários realistas por profissionais de segurança ofensiva, como os *Pen-testers*, profissionais de segurança ou *hackers* éticos que simulam os ataques cibernéticos para encontrar vulnerabilidades no sistema computacional (??), inclusos no *Red Team* da empresa, que reúne todos os profissionais autorizados a reproduzir investidas no sistema como se fossem um potencial adversário contra a postura de segurança do ambiente (??). O estudo de ??) também utiliza a engenharia de *prompt* para extrair melhores informações das LLMs e automatizar tarefas correlacionadas, através da exploração do uso ofensivo das LLMs (para fins de conhecimento). Em contrapartida, o presente projeto sugere o uso de forma defensiva, voltado à detecção de ameaças e correlação de eventos.

4 MATERIAIS E MÉTODOS

4.1 Materiais

Os materiais utilizados para o desenvolvimento deste trabalho serão essencialmente as regras *Sigma* (arquivos em YAML) disponíveis em seu repositório (??) — composto atualmente por mais de 3.400 regras; as LLMs mais conhecidas e de ampla utilização, como *ChatGPT*, *Deepseek*, *Microsoft Copilot*, *Gemini* e *Grok*; linguagem de programação *Python*; ambiente de desenvolvimento *Visual Studio Code*; para o desenvolvimento do agente, *framework* de código aberto *LangChain* — facilitador da construção de aplicações que usam modelos de linguagem grandes (LLMs) (??).

4.2 Métodos

A primeira fase envolveu a geração manual de regras utilizando as LLMs já mencionadas, visando um cenário simplificado e que remete a um usuário inexperiente. A segunda fase incluirá um *prompt* customizado que fornecerá dados e instruções relevantes para extrair melhor desempenho e acurácia das LLMs na geração das regras. Já na terceira fase será desenvolvido um agente especializado, treinado para a geração de regras de forma autônoma, com base nas referências/dados da vulnerabilidade da qual se pretende proteger.

Inicialmente serão obtidas todas as regras *Sigma* por meio de seu repositório no *GitHub* (??), sendo organizadas da seguinte forma: 3000 regras para a base de treinamento e 400 regras para testes. A avaliação dos resultados será realizada através de análise manual por amostragem e análise automatizada (Distância do Cosseno, *Term Frequency-Inverse Document Frequency* (TF-IDF), Jaccard - à definir) para verificar a acurácia perante as regras originais ou eventual melhoria em sua estrutura, por se tratar de uma busca mais recente, utilizando informações adicionais às referências usadas durante a criação da regra original. Por fim, uma análise de qual fase alcançou o melhor desempenho e qualidade será apresentada.

As etapas de desenvolvimento e avaliação do presente trabalho são detalhadas a seguir.

4.2.1 Primeira fase

Inicialmente serão selecionadas manualmente 10% das regras de teste (40); elas serão utilizadas para a geração de regras através de um *prompt* simples, sem customizações, em cada uma das LLMs, da mesma forma que um usuário inexperiente faria, conforme é exibido no “**Prompt Básico**”. Para solicitar a geração das regras, as referências de cada uma delas serão extraídas e armazenadas, já que contém o(s) *link*(s) com as informações fundamentais que

deram origem àquela regra. Em seguida serão incluídas no *prompt* e enviadas como requisição para a IA de forma manual.

Prompt Básico

“Generate a SIGMA rule (<https://sigmahq.io/docs/basics/rules.html>) for this event:
<https://localtonet.com/documents/supported-tunnels>
<https://cloud.google.com/blog/topics/threat-intelligence/unc3944-targets-saas-applications>”

Para fins explicativos, os *links* posteriores à palavra “event” são as referências inseridas na regra *Sigma* pelo seu autor (visíveis no campo “reference” das regras); e o *link* mencionado após a palavra “rule” é a página do repositório que instrui a escrita de uma regra *Sigma*.

O próximo passo será gerar as regras de forma automatizada, utilizando-se de um *script* para otimizar o processo. Tanto a seleção manual quanto a automatizada constituirá os resultados deste trabalho.

Após reunir réplicas¹ de regras *Sigma* já existentes, será realizada comparação manual por amostragem entre a regra original e suas réplicas, considerando os campos mais relevantes (que tornam a regra ativa ou operável no SIEM); são eles: os componentes *detection* e *logsource*.

Ao final dessa análise, o estudo demonstrará a qualidade das regras *Sigma* escritas através de um *prompt* básico e será possível indicar qual LLM se destacou percentualmente perante as demais.

4.2.2 Segunda fase

A segunda fase utilizará engenharia de *prompt* para fornecer às LLMs melhor contexto e informações precisas que aprimorem as regras desenvolvidas por elas, novamente resultando num estudo comparativo ranqueando quais devolvem melhores resultados.

O processo envolve o uso estratégico de palavras-chave, clareza na formulação dos textos, além de adaptação do *prompt*(??), pois segundo a própria ??, “diferentes tipos de modelos de linguagem podem precisar ser solicitados de forma diferente para produzir os melhores resultados”.

O processo de amostragem e teste da primeira fase será replicado, porém com o diferencial do refinamento dos *prompts*.

Sendo assim, as mesmas 40 regras de testes selecionadas serão utilizadas para a geração de regras através de um *prompt* customizado, em cada uma das LLMs – aqui o usuário é mais experiente que o anterior. Em seguida, utilizar-se-á de um *script* que automatize a tarefa.

¹ As réplicas são as regras geradas pelas IAs.

É importante salientar que em todas as fases será inclusa uma averiguação final de um especialista e/ou da “comunidade *Sigma*” (pessoas que contribuem regularmente com o repositório de regras). Ademais, os respectivos resultados passarão por análise qualitativa novamente e o método de comparação usado na primeira fase será repetido.

4.2.3 Terceira fase

Na terceira fase do projeto, o agente de IA será projetado e testado segundo sua precisão e qualidade das regras geradas. O desenvolvimento do agente começará por escrita de código e, quando preparado, será treinado com 3000 regras *Sigma*; para o teste serão 400. Ajustes serão feitos conforme necessidade. Das 400 réplicas por LLM (serão 2 mil no total, se forem usadas 5 LLMs), 10% serão retiradas para uma análise inicial manual, posteriormente automatizada.

Para as avaliações, primeiro será necessário definir parâmetros que validam uma regra. De antemão já pode-se considerar que as regras validadas serão, no mínimo, iguais às originais – o que não significa exatamente que são boas, pois algumas podem já estar datadas, por exemplo –, fazendo-se necessário verificar as regras descartadas para entender o motivo da não validação, pois algumas regras diferentes podem ser melhores que as originais. A avaliação das regras descartadas será feita por especialista ou pela comunidade *Sigma*.

Com os critérios definidos, os desempenhos dos LLMs serão medidos utilizando métricas qualitativas e quantitativas, calculadas para cada modelo e fase do experimento. Serão calculados, por exemplo, taxa de similaridade média textual, taxa de acerto e desvio padrão, analisando a proporção de réplicas consideradas “semelhantes” à original.

O produto final do trabalho será uma composição analítica qualitativa e quantitativa entre todos os produtos obtidos, das três fases.

5 RESULTADOS

Este capítulo apresenta o que foi obtido como resultado do trabalho, que, em princípio, é o sistema desenvolvido. Se não for um sistema, como, por exemplo, uma solução na área de redes, neste capítulo é reportada a solução proposta. Neste caso, a divisão do capítulo em seções é realizada, se necessária, de acordo com o trabalho.

O capítulo pode conter seções de acordo com o tipo de sistema e a necessidade de documentação mais extensa de determinados aspectos. Caso o trabalho se refira à comparação entre tecnologias ou dados obtidos como resultados do uso do sistema, além da descrição do sistema, há os dados obtidos com os testes e a discussão desses dados. Nesse caso haverá uma seção para os dados obtidos desses testes e as discussões.

5.1 Escopo do sistema

Apresenta o escopo do sistema (contendo entre dois ou cinco parágrafos) de forma bastante sucinta, considerando aspectos técnicos e conceituais. O escopo define o que é o sistema, consistindo das funcionalidades e características que o sistema deve conter. É importante apresentar também o escopo negativo, ou seja, as funcionalidades e características que o sistema não irá conter. Exemplo:

O sistema XYZ deve gerenciar todos os processos de uma livraria virtual, desde a aquisição até a venda dos livros para o consumidor final. O acesso dos compradores e gerentes deve ser feito por meio de um site WEB, incluindo a possibilidade de acesso por outras tecnologias (ex. celular, tablet). Os clientes poderão fazer as compras pagando com cartão de crédito ou depósito bancário. Existem promoções eventuais pelas quais os livros podem ser comprados com desconto.

De início, a livraria vai trabalhar apenas com livros novos a serem adquiridos de editoras que tenham sistema automatizado de aquisição. Desta forma, o sistema a ser desenvolvido deve conectar-se aos sistemas das editoras para efetuar as compras.

O sistema deve calcular o custo de entrega baseado no peso dos livros e na distância do ponto de entrega. Eventualmente podem haver promoções do tipo “entrega gratuita” para determinadas localidades.

O sistema deve permitir a um gerente emitir relatórios de livros mais vendidos, e compradores mais assíduos, bem como sugerir compras para compradores baseadas em seus interesses anteriores.

5.2 Modelagem do sistema

A modelagem do sistema inclui os diagramas e as descrições textuais para representar o problema e a solução.

Sendo assim, primeiramente esse item deve apresentar diagramas utilizados para a modelagem de negócios (ex. diagramas de atividade e estado), se esses tenham sido necessários. Em seguida esse item deve conter a descrição dos requisitos obtidos do usuário, contendo sua respectiva classificação (funcionais e não funcionais). Sugere-se o uso de um modelo formal sugerido por autores (ex. Wazlawick, Bezerra) para a apresentação dessa classificação.

Se utilizada orientação a objetos e a UML, nesta seção ainda são apresentados, por exemplo, os diagramas de casos de uso, com suas descrições suplementares, os diagramas de classe de análise (ou modelo conceitual), de sequência e/ou comunicação, diagrama de classes de projeto.

Nesta seção também estão os diagramas da modelagem de banco de dados, como entidade-relacionamento. Nesse item pode ser apresentada a descrição de cada uma das classes do modelo de classes apresentado acima, assim como a descrição das tabelas do banco de dados. Também podem estar documentados modelos e padronizações utilizados para a interface, diagramas de navegação, a representação da arquitetura do sistema e dos padrões de projeto utilizados.

5.3 Apresentação do sistema

Apresenta as funcionalidades e o uso de recursos tecnológicos do sistema por meio de suas telas, enfatizando a interação com o sistema. A apresentação do sistema é feita sob a forma de texto, com telas e definição de padrões que forem relevantes ao contexto do trabalho. As telas são tratadas como figuras, cópias (print screen) de relatórios ou consultas também são figuras.

A ?? exibe a tela de acesso ao Cadastro de Pacientes.

Figura 3 – Tela de acesso ao Cadastro de Pacientes.

Fonte: Autoria própria (2026).

5.4 Implementação do sistema

Nesta seção é documentada a implementação do sistema com partes relevantes ou exemplos de código, rotinas, funções. Inclui, ainda, a descrição técnica do uso de recursos (componentes, bibliotecas, etc.) da linguagem. Ressalta-se que cada orientador avaliará juntamente com seu orientado o que poderá ser descrito nesta seção. Isso sem que sejam revelados

detalhes do sistema que possam comprometer seu uso comercial ou científico ou que a descrição fique muito sucinta ou superficial.

Em materiais e método estão quais os recursos utilizados, neste capítulo é reportado como esses recursos foram utilizados para resolver o problema.

Sugere-se colocar listagens curtas de código, enfatizando aspectos específicos das tecnologias utilizadas ou da implementação. Sugere-se, ainda, que o código não seja apresentado sob a forma de print screen, e sim copiado e colado no texto, mantendo, se possível, a formatação. Todas as listagens de código devem ser devidamente explicadas. A explicação deve ser técnica, fundamentada em aspectos conceituais e boas práticas de programação.

Enfatizar os diferenciais do sistema: procedimentos armazenados, consultas SQL, uso de componentes, uso de padrões de projeto, a forma de uso dos recursos da linguagem. Esses diferenciais são no sentido de explicitar as vantagens, desvantagens, dificuldades e facilidades que esses recursos impetraram no desenvolvimento do sistema em termos técnicos. Esses diferenciais servirão para avaliar pela utilização ou não desses recursos, pelo menos para sistemas iguais ou semelhantes ao reportado no trabalho.

Reportar a forma como o sistema foi verificado e validado. No sentido de verificar se os requisitos definidos para o mesmo foram atendidos. Os testes podem ser realizados pelo professor orientador, pelos professores que compõem a banca, por pessoas que serviram de base para as informações para o sistema e etc. Os testes podem ser realizados com base em um plano de testes elaborado juntamente com a análise e projeto do sistema. Para validar a implementação podem ser desenvolvidas rotinas de teste unitário.

Se houver implantação do sistema, mesmo que seja para teste, reportar a forma como isso foi feito, a geração de instaladores, os problemas com ambiente e sistema operacional, incluindo banco de dados e outros. Deixar explícito o procedimento para instalar e usar o sistema.

Quando for necessário, citar no texto do trabalho nomes de campos, tabelas ou rotinas específicas utilizadas na implementação de um software, utilizar a fonte courier new para destacar esses nomes.

Um exemplo de listagem de código fonte pode ser observado na ??, que representa a classe Aluno.

5.5 Discussões (opcional)

O trabalho contém esta seção quando considerado que há resultados (em termos de dados) e discussões relevantes ou suficientes para justificar uma seção. Se existentes e não justificarem uma seção, eles podem estar na seção que relata a implementação do sistema.

Nesta seção estão os resultados obtidos da realização de testes quantitativos e qualitativos, independentemente da quantidade, tipo e volume de testes realizados. Os resultados dos testes são discutidos tendo como base o referencial teórico e os objetivos pretendidos com o trabalho. Esses testes podem resultar de implantação e testes de uso do sistema.

Listagem 1 – Classe Aluno

```
1 @Entity
2 public class Foo {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String nome;
9
10    private Integer ra;
11
12    // constructor, getters and setters
13 }
```

Fonte: Autoria própria (2026).

6 CONCLUSÃO

Inicia com um resumo do trabalho, retomando o(s) objetivo(s), o referencial teórico e o uso das ferramentas e das tecnologias utilizadas no trabalho.

A conclusão contém a opinião do autor em relação às vantagens, desvantagens, facilidades e limitações das tecnologias e/ou do método utilizados, as dificuldades encontradas e como foram superadas.

Também devem ser apresentadas as vantagens, desvantagens e limitações do trabalho desenvolvido, sempre tendo em vista a sua contribuição para a comunidade acadêmica e profissional e para a sociedade como um todo.

É a opinião técnica do autor do trabalho em relação ao assunto sob a forma de uma espécie de avaliação em relação ao trabalho desenvolvido e as tecnologias utilizadas.

Finaliza verificando se o objetivo foi alcançado e com a opinião do autor sobre o assunto, de acordo com o referencial teórico e com os resultados obtidos.

As perspectivas futuras são opcionais, devem ser apresentadas somente caso o acadêmico pretenda dar continuidade ao trabalho, ou mesmo se ele julgar relevante que outras pessoas dêem continuidade ao seu trabalho.

